幾何音響シミュレーションの高速化（2026-08-19〜23）

最終更新: 2026-08-23（HBN240018）

チーム共有用の説明資料。**何が遅かったのか／何を変えたのか／どれだけ速くなったのか**を、 順を追って説明する。数字はすべて実案件のモデル（研修室・視聴覚室・階段教室）での実測値。

---

0. まず結論

施策	効果	副産物
① 同一平面パッチ単位の交差判定	判定 **2.3〜3.0 倍**	**受音経路が 2.2 倍**（取りこぼしが直った）
② バックトレースの配列演算化	**28.7 倍**	—
③ 音線追跡を受音点間で共有	受音点の数だけ効く	受音点を増やす費用がほぼゼロに
④ インパルス応答を時間領域で合成	**20〜65 倍**	11.3 kHz まで位相ごと一致
⑤ 受音点に依らない計算を 1 回に	統計残響式ぶん	ログが読みやすくなった
通し（受音点 5 点）	**10.7 倍**（①〜③）／さらに ④⑤ で **1.7 倍**	結果は従来と一致

実案件の計算時間：**1 時間 45 分 → 10 分強**。

---

1. どこが遅かったのか

計算は大きく 3 段階に分かれる。

```
①音線追跡          音源から音線を飛ばし、受音球を通った経路の「反射面の並び」を記録
②重複削除          同じ反射面の並びを持つ経路をまとめる
③バックトレース    虚音源法で経路を検証し、到来時刻とエネルギーを出す
```

手を入れる前の内訳（研修室・音線 10,000 本・最大反射 160 回）。

```
①音線追跡          15.22 秒  (34.4%)
②重複削除           0.09 秒  ( 0.2%)
③バックトレース    28.89 秒  (65.4%)
```

③が 2/3 を占めていた。そして①③はどちらも受音点ごとに丸ごとやり直していたので、 受音点 5 点なら全部が 5 倍かかっていた。

2. 施策① 同一平面パッチ単位の交差判定

きっかけ

> 三角形要素で分割しているのは、あくまでもねじれのない面を定義する必要があるため、と思っています。
> ねじれの無い平面であれば、多頂点要素でも良いのでは？

そのとおりだった。**三角形に割るのは「ねじれのない面」を保証するためであって、 判定の都合ではない。**

やったこと

同一平面に並んだ三角形をまとめて **1 枚の多角形（パッチ）** として判定する。

```
ModelTest  三角形  68 枚 → パッチ  16
階段教室   三角形 272 枚 → パッチ  70
視聴覚室   三角形 292 枚 → パッチ 114
```

判定は「平面との交点を 1 回求め、外周多角形の内側かを**交差数の偶奇**で数える」。 外周は輪に並べ替えず**辺の集まりのまま**扱うので、穴の開いた面もそのまま正しい。

パッチにまとめる条件は 4 つとも満たすこと。

1. 同一平面
2. 辺を共有して連結している
3. **同じ材料**（まとめると吸音率が引けなくなる。視聴覚室では扉・窓が壁と同じ平面で

   接しており、同一平面だけでまとめると 18 パッチで材料が混ざった）
4. 同じ法線の向き

★同一平面のしきい値は編集画面より厳しくする（0.1° ／0.1 mm）。 編集画面の 1° ／1 mm は「壁 1 枚」を選びやすくするための値で、判定に使うと**形が変わる。** 実測（階段教室）：緩い値だと代表面の平面から 3.5 mm ずれた三角形まで同じパッチに入り、 交点距離が 3.15 mm 動いた。

効果

モデル	判定の速度
ModelTest	3.0 倍
階段教室	2.6 倍
視聴覚室	2.3 倍

いずれも三角形版と**距離・交点がビット単位で一致**する。

★速度より大きかった副産物

通しで回すと、こうなった。

```
視聴覚室 / 音線 30,000 本 / 最大反射 40 回
  三角形  62.4 秒  経路 14633 本 → 受音  314 本
  パッチ  31.2 秒  経路 14234 本 → 受音  703 本   ← 2.2 倍
```

受音できる経路が 2.2 倍に増えた。

理由：バックトレースは「記録した反射面の並びを再現できるか」で経路を検証する。 三角形単位だと、**同じ壁の隣の三角形に当たっただけで却下**していた。 鏡像は平面で決まるのでこれは純粋な取りこぼしで、パッチ単位にすると正しく通る。

後期の経路が薄く残響時間が信用できないという問題にも効いている。

---

3. 施策② バックトレースの配列演算化

何が遅かったか

経路 1 本ずつ Python のループで回り、1 経路につき反射回数ぶん交差判定を**単発で** 呼んでいた（研修室で経路 37,592 本 × 最大 160 回）。

やったこと

全経路を同時に、反射回数 k を大きいほうから 1 段ずつ下ろす。

経路の長さはまちまちだが、「長さ L の経路は k = L のときに入ってくる」と考えると、 **どの時点でも動いている経路はすべて同じ k にいる**ので素直に束で処理できる。

```
k = Kmax     長さ Kmax の経路が入る
k = Kmax-1   長さ Kmax-1 の経路が入り、上の経路は 1 段下りている
   …
k = 0        直接音の区間。ここまで残った経路が受音成立
```

虚音源も同じく束で作る（面での鏡像は `p - 2(n·p + d)n` の 1 行で、経路ごとに面が 違うだけなので配列で引ける）。

効果

```
研修室 / 音線 10,000 本 / 160 反射 / 経路 37,592 本
  1 本ずつ 31.54 秒 → 束 1.10 秒    （28.7 倍）
  受音した経路は 864 本で同じ、エネルギーの差は倍精度の丸め（3e-16）
```

1 本ずつの実装（`backtrace_path`）は**参照実装として残してある。** 一致することをテストで確かめている。

---

4. 施策③ 音線追跡を受音点間で共有

きっかけ

> 今複数の受音点があった場合、1 個の受音点に対して音線追跡していますか？
>
> 一回の音線追跡で受音判定はできないですか？

コードを読むと、**追跡は受音点に依存していなかった。**

```
inside = rs.inside_sphere_batch(sphere_radius, direction_alive, origin_alive,
                                receiver_point, min_distance)
for local in np.nonzero(inside)[0]:
    results.append(history[ray, :depth[ray]].tolist())   # 記録するだけ
```

受音しても音線は打ち切られない（終了条件は「当たる壁が無い」か「最大反射に達した」だけ）。 反射も履歴も受音点と無関係に進む。つまり受音球を複数置いて同時に判定すれば、 追跡は 1 回で済む。

やったこと

`loop_reflectionmesh.loop()` が受音点を複数受け取り、受音球の判定だけ受音点の数ぶん行う。
`run_project` は複数受音点のとき 1 回だけ追跡して各受音点に配る。

効果

```
研修室 / 音線 6,000 本 / 160 反射 / 受音点 5 点
  旧 142.0 秒 → 新 13.2 秒    （10.7 倍）
  受音経路の本数・バンド別エネルギーとも全 5 点で完全一致
```

②と合わせた結果。段階別の内訳はこう変わった。

```
          以前                          いま
音線追跡        34%（×受音点数）   →  1 回で共有
バックトレース   65%（×受音点数）   →  28.7 倍速く、受音点ごと
```

受音点を増やす限界費用がほぼゼロになった。5 点でも 20 点でも追跡の費用は変わらない。

4.5 インパルス応答の合成を時間領域に（2026-08-21）

> インパルス応答の合成が、1 受音点毎にやってるのが、遅くないかな？とか思ってます。

そこだった。 段階別に測ると、受音点 5 点で

```
インパルス応答の合成    14.39 秒（全体 31.1 秒の 46%）
重複経路の削除           2.92 秒
残響時間の算出           0.27 秒
統計残響式               0.04 秒 ← 受音点ごとに回っていた（無駄）
```

中身をさらに測ると、**伝達関数（式2.67）の 15.7 秒**が全部で、FFT は 0.18 秒だった。 式(2.67) は**周波数の数 × パルスの数**（70,313 × 8,825）の総当たりになる。

時間領域に置いてから FFT すれば「パルスの数 + FFT」で済む。 到来時刻がサンプルの格子に乗らないぶんは**窓付き sinc（64 タップ）**で分配する。

```
研修室・受音点 1 点・パルス 8,825 本・応答 3 秒
  厳密（式2.67 をそのまま） 15.54 秒
  高速（時間領域 → FFT）      0.24 秒    （65 倍）
  T30 の差 0.0000 s / C50 の差 0.000 dB / 波形の相関 0.998
```

通しでは **31.1 秒 → 17.9 秒**（受音点 5 点。統計残響式を 1 回にした分も含む）。 厳密版は `method="exact"` で使えるよう残してある。

★これは「時間領域で足す」ことではない（2026-08-23 の確認）

査読論文 Toyoda & Sakayoshi (2021) 第 3 節は、まさにこれを警告している。 ただし**問題になるのは「到来時刻をサンプル時刻へ寄せる」ことで、時間領域そのもの ではない。**寄せると位相が半サンプルずれ、**干渉の谷の位置が動く。**

こちらは寄せず、端数を窓付き sinc（＝理想遅延を帯域制限して切り詰めたもの）で 本来の位置に置いている。谷の位置を測ると

やり方	干渉の谷	11.3 kHz までの振幅差 / 位相差
厳密（式(2)）	5937.8 Hz	—
窓付き sinc（本実装）	**5937.8 Hz（一致）**	**0.00001 dB / 0.00002°**
サンプル時刻へ寄せる	5512.5 Hz（7% ずれる）	4.6 dB / 48.8°

タップ数を増やすと単調に厳密解へ寄る（64→0.99744、256→0.99975、512→0.99988）。 既定は **256 タップ**（0.52 秒。厳密版 15.9 秒の **31 倍**速い）。 足し込みを `np.bincount` にして（`np.add.at` から）**結果を変えずに 1.9 倍**にしたので、 タップ数を増やす負担が小さい。**速さと厳密さのどちらを取るかで悩まなくてよい。**

波形や伝達関数そのものを波動解と突き合わせるときは厳密版を使う（設定画面の「インパルス応答の合成」で選べる）。

あわせて、**受音点に依らない計算を受音点ごとに回さない**ようにした。 統計残響式は 1 点目の結果を 2 点目以降へ配る（ログも 1 回だけになる）。

5. 複数音源への対応（PA システムなどを想定）

受音点と違って共有はできない

音線は音源から出るので、音源が違えば別の音線が要る。追跡の費用は音源の数に比例する。

ただし配列演算の効率は活かせる

音源ごとの音線を 1 本の配列に並べて追跡するようにした（`loop()` が音源も複数受け取る）。 どの音源から出た音線かは添字で持ち、戻り値は `[音源][受音点]` の入れ子になる。

束が大きくなるぶん配列演算の効率はむしろ上がる（音源ごとに呼び直すと、 そのたびに小さい束で回すことになる）。

残っている作業（設計）

追跡までは入ったが、**パイプライン全体はまだ 1 音源前提**。必要な作業は次のとおり。

1. **バックトレースを（音源, 受音点）の組ごとに回す。**

   虚音源は音源に依存するので組ごとに必要。ただし②で束処理になっているので費用は小さい。
   組の数は 音源数 × 受音点数。
2. **パルス列の合成方法を決める。**
   - **非干渉（エネルギー和）** … 音源ごとのエネルギーをバンドごとに足す。ｐｒｏｓｔｏｊ。
   - **干渉（インパルス応答の和）** … 音源ごとの遅延・利得を掛けてから時間軸で足す。

     PA はこちらが要る（ディレイタワーの時間差、レベル差、指向性）。
     `impulse.impulse_response_from_pulses` はパルス列を受け取るので、
     音源ごとのパルス列に遅延と利得を掛けて 1 本にまとめてから渡せばよい。
3. **音源ごとの属性を持つ。** 利得 [dB]、遅延 [ms]、指向性。

   `project.json` に音源の配列として持たせる（いまは 1 点の座標だけ）。
4. **出力の整理。** 受音点ごとに「音源別の寄与」と「合計」を出せると設計に使える。

DXF 側は `src` レイヤに POINT を複数置けば読める（`DxfModel.source_points` は すでに複数を保持している。使っているのが 1 点目だけ）。

---

6. 音線の経路は保存されているか（受音点を後から足せるか）

保存されていない。 `raylog.npz` に入るのは可視化用に間引いた 2,000 本だけで （20 万本から 100 本おき）、本線の反射面 ID 履歴はメモリ上だけで捨てられる。

全部を保存すれば追跡を省けるが、音線 20 万本 × 300 反射 × 節点座標で **1 GB 近く**になる。

一方、施策③で追跡が共有できるようになったので、**受音点は最初に多めに置くのが いちばん安い。**追跡の費用は受音点の数に依らない。

後から足したい場合の限界費用は「追跡 1 回ぶん + その受音点のバックトレース」。 研修室（20 万本・300 反射）で約 8 分。

---

7. 検証の方針

速くなったが答えが変わった、では意味がない。すべての施策で、変更前の実装を 参照実装として残し、一致することをテストで確かめている。

施策	参照実装	確認内容
① パッチ判定	`mesh_method.FaceArrays`（三角形単位）	当たり外れ・距離・交点がビット単位で一致
② 束バックトレース	`loop_noredundancy.backtrace_path`（1 本ずつ）	残る経路・時刻・方向が一致、エネルギーは丸め（1e-12 未満）
③ 追跡の共有	受音点ごとの追跡	経路のリストが完全一致
複数音源	音源ごとの追跡	同上

さらに、通しの計算でも受音経路の本数とバンド別エネルギーが全受音点で一致することを 実データで確認した。

テストは **535 件**（`.venv\Scripts\python tests\test_geosim.py`）。

---

8. 用語

用語	意味
音線追跡	音源から音線を飛ばし、壁で反射させながら受音球を通るか調べる
受音球	受音点を中心とする球。音線が通れば「受音した」とみなす
バックトレース	虚音源法。反射面の並びから鏡像を作り、その経路が本当に成立するか検証する
虚音源	音源を壁で鏡映した点。反射経路を直線として扱うための道具
面グループ／パッチ	同一平面で連結した三角形のかたまり。設計者が描いた「1 枚の壁」に相当
スリバー	細長い三角形。数値的に不利になりやすい